

# Laboratorium przetwarzanie obrazów

Radosław Piotrowicz

Informacje potrzebne do implementacji

## Spis treści

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Gdzie znajdują się interesujące nas rzeczy.</b>    | <b>3</b> |
| 1.1      | Laby 1 . . . . .                                      | 3        |
| 1.2      | Laby 2 . . . . .                                      | 3        |
| 1.3      | Laby 3 . . . . .                                      | 3        |
| 1.4      | Laby 4 . . . . .                                      | 3        |
| 1.5      | Laby 5 . . . . .                                      | 4        |
| 1.6      | Laby 6 . . . . .                                      | 4        |
| <b>2</b> | <b>Co jest potrzebne przy implementacji algorytmu</b> | <b>4</b> |
| 2.1      | Opcjonalne elementy . . . . .                         | 4        |
| <b>3</b> | <b>Opis klas i przydatnych metod</b>                  | <b>5</b> |
| 3.1      | Klasa Algorithm . . . . .                             | 5        |
| 3.2      | Klasa MorphologicAlgorithm . . . . .                  | 6        |
| 3.3      | Klasa Image . . . . .                                 | 7        |
| 3.4      | Struktura Pixel . . . . .                             | 8        |

## 1 Gdzie znajdują się interesujące nas rzeczy.

W każdym pliku z algorytmem należy zaimplementować metodę **AlgorithmFunction**. Wyjątek stanowią algorytmy z labów 4. Tam implementacja wygląda trochę inaczej.

**Jeżeli jakiś algorytm zawiera parametry, to znajdują się one w pliku nagłówkowym <nazwa>.hpp.**

Ponadto Klasa **Algorithm**, po której dziedziczą wszystkie algorytmy zawiera kilka metod pomocniczych, z których można korzystać.

### 1.1 Laby 1

- Negative.cpp
- Brighten.cpp
- Contrast.cpp
- Logarithm.cpp
- Exponentiation.cpp
- LeveledHistogram.cpp
- Masking.cpp
- Mixing.cpp

### 1.2 Laby 2

- Binarization.cpp

### 1.3 Laby 3

- LinearFilter.cpp
- MedianFilter.cpp

### 1.4 Laby 4

**UWAGA** tutaj jest lekka zmiana w implementacji funkcje erozji i dylatacji znajdują się w **MorphologicAlgorithm.cpp** (mogą zostać potem wykorzystane przy otwarciu i zamknięciu)! Pozostałe algorytmy implementujemy normalnie.

- **MorphologicAlgorithm.cpp** (tutaj należy uzupełnić metody **ErosionFunc** oraz **DilatationFunc**)
- **Erosion.cpp** (pomijamy)
- **Dilatation.cpp** (pomijamy)

- Opening.cpp
- Closing.cpp
- ContourInner.cpp
- ContourOuter.cpp

## 1.5 Laby 5

- Skeletonization.cpp

## 1.6 Laby 6

- Hought.cpp

# 2 Co jest potrzebne przy implementacji algorytmu

Każdy algorytm jest wykonywany w oddzielnym wątku, aby program był responsywny w związku z czym. Każdy algorytm musi mieć w sobie 2 wywołania:

- CopyToLocalVariable - na początku (kopiuje obraz do lokalnej zmiennej copy)
- SaveToOutput - na końcu (zapisuje lokalną kopię z powrotem)

**W każdym algorytmie są one umieszczone proszę swój kod umieścić pomiędzy!**

## 2.1 Opcjonalne elementy

Można też opcjonalnie dodać odświeżanie i możliwość przerywania wykonywania. **Natomiast nie jest to wymagane.**

Automatyczne odświeżanie można włączyć w zakładce Ustawienia. Tam również można ustawić co ile sekund ma się odświeżać /newline Na obrazku 1 przedstawione jest jak korzystać z opisanych tu funkcji.

```

... CopyToLocalVariable(outputImage);

... for (int row = 0; row < copy.GetHeight(); row++)
... {
...     for (int col = 0; col < copy.GetWidth(); col++)
...     {
...         // TUTAJ IMPLEMENTACJA ALGORYTMU
...     }
...     if (Canceled(outputImage))
...         return;
...     AutomaticRefresh(outputImage);
... }
... SaveToOutput(outputImage);

```

Rysunek 1: Przykład wykorzystania

## 3 Opis klas i przydatnych metod

### 3.1 Klasa Algorithm

Jest to klasa, na której bazują wszystkie algorytmy, przydatne metody w tej klasie to:

- void ParamsMenu() - metoda zawierająca elementy menu do ustawienia parametrów.
- void AlgorithmFunction(Image \*outputImage) - **funkcja algorytmu tutaj uzupełnić kod**
- void ResetToDefaults() - restart parametrów do wartości bazowej
- void Save(std::ofstream& file) - zapis parametrów do pliku (opcjonalne)
- void Load(std::ifstream& file) - wczytanie parametrów z pliku (opcjonalne)
- void CopyToLocalVariable(Image \*outputImage) - **musi się znaleźć w funkcji algorytmu na początku**
- void SaveToOutput(Image \*outputImage) - **musi się znaleźć w funkcji algorytmu na końcu**
- bool Canceled(Image \*outputImage) - używamy, jeżeli chcemy móc przerwać wykonywanie algorytmu (opcjonalne)
- void AutomaticRefresh(Image \*outputImage) - używamy, jeżeli chcemy móc odświeżyć obrazek wyjściowy w trakcie przetwarzania (opcjonalne)
- void ManualRefresh(Image \*outputImage) - podobnie jak metoda powyżej, ale tutaj program nie będzie robił tego automatycznie po włączeniu opcji (opcjonalne)

Zawiera również takie zmienne jak:

- bool autoRefresh - informuje program w jaki sposób odświeżyć (automatycznie - program ma to zrobić, ręcznie - programista decyduje kiedy)
- std::string algorithmName - nazwa algorytmu wyświetlana w menu wyboru algorytmów
- Image copy - **lokalna kopia obrazu, na której pracujemy**

### 3.2 Klasa MorphologicAlgorithm

Jest to klasa pośrednia, na której bazują algorytmy z labów 4. Nowe metody to:

- void CalculateOffsets() - jeżeli chcemy aby element przeszedł po obrazie jak najdokładniej, można policzyć jego maksymalną odległość od każdej granicy ta metoda to robi (opcjonalne)
- void ErosionFunc(Image \*outputImage) - **tutaj implementowana będzie erozja** (może być ponownie wykorzystane przy otwarciu i zamknięciu)
- void DilatationFunc(Image \*outputImage) - **tutaj implementowana będzie dylatacja** (może być ponownie wykorzystane przy otwarciu i zamknięciu)

Użyteczne nowe zmienne to:

- int elementSize - rozmiar elementu strukturalnego
- bool element3x3[3][3] - element o wymiarach 3x3
- bool element5x5[5][5] - element o wymiarach 5x5
- bool element7x7[7][7] - element o wymiarach 7x7
- Image copyRead - **kopia do odczytu, bo są to operacje kontekstowe**
- int32\_t offsetLeft = 0 - minimalna odległość od lewej granicy obrazu, obliczone przez metodę **void CalculateOffsets()**
- int32\_t offsetRight = 0 - minimalna odległość od prawej granicy obrazu, obliczone przez metodę **void CalculateOffsets()**
- int32\_t offsetTop = 0 - minimalna odległość od górnej granicy obrazu, obliczone przez metodę **void CalculateOffsets()**
- int32\_t offsetBottom = 0 - minimalna odległość od dolnej granicy obrazu, obliczone przez metodę **void CalculateOffsets()**
- int32\_t elemntCopy[7][7] - kopia elementu może być wykorzystane w głównej funkcji **Ustawiana jest w metodzie void CalculateOffsets()**

### 3.3 Klasa Image

Klasa Image jest kontenerem na obrazy, które będziemy przetwarzać.

Najbardziej interesuje nas w niej SDL\_Surface, czyli struktura z informacjami o pikselach w obrazie.

**UWAGA, jeżeli jakaś metoda ma wersję z końcówką NoTexture to proszę korzystać z tej wersji z tą końcówką!**

Zawiera takie przydatne metody jak:

- void CopyNoTexture(Image &other) - kopiuje zawartość danego obrazu
- int32\_t SetSourceImageNoTexture(std::filesystem::path path) - ustawia zawartość image na obraz o podanej ścieżce (lepiej jest korzystać z menu do wczytania)
- Pixel GetPixel(int col, int row) - zwraca Strukturę Pixel z informacjami o pikselu w danym punkcie
- void SetPixel(int col, int row, Pixel pix) - ustawia pixel w danym punkcie na podany
- void SetPixelWhite(int col, int row) - ustawia pixel w danym punkcie na kolor biały
- void SetPixelBlack(int col, int row) - ustawia pixel w danym punkcie na kolor czarny
- int GetWidth() - zwraca szerokość obrazu
- int GetHeight() - zwraca wysokość obrazu
- int GetPixelCount() - zwraca ilość pikseli
- float\* GetLightValues() - zwraca tablice float o długości 256 z wartościami jasności (są też wersje dla poszczególnych składowych)
- float\* GetDistributor() - zwraca tablice float o długości 256 z wartościami dystrybucyjnymi (są też wersje dla poszczególnych składowych)
- void CopyNormalisedBrightnessHistogram(float \*dst) - kopiuje histogram (unormowany do wartości 0.0 - 1.0) do podanej tablicy
- bool NoSurface() - czy obraz zawiera poprawnie zainicjowane SDL\_Surface (jeśli nie to źle)
- void SetBlankSurfaceNoTexture(int width, int height) - ustaw obraz na biały prostokąt o wymiarach
- void RefreshPixelValuesArrays() - odśwież tablice z histogramami i dystrybucyjnymi (po poprawnym zakończeniu wątku z algorytmem wątek główny automatycznie odświeży)

### 3.4 Struktura Pixel

Ta struktura zawiera wartości R, G, B oraz jasność piksela.

**Należy pamiętać, że gdy modyfikujemy piksel, należy ustawić wszystkie 3 składowe.**

**WARTOŚĆ JASNOŚCI JEST TYLKO DO ODCZYTU. METODA SetPixel NIE BIERZE JEJ POD UWAGĘ.**